

CONTENTS

Divide y Vencerás	
Guía de estudio con tolerancia cero — Módulo 5, Programación 3 (FING UdelaR)	
Fuente: Kleinberg & Tardos, Algorithm Design, cap. 5 (+ referencias a 13.5) — Prácticos semana 7 y 8	
0. Por qué existe este tema	
1. Esquema general: dividir, resolver, combinar	
2. Cómo plantear la recurrencia de tiempo $T(n)$	
3. El Teorema Maestro	
3.1 Enunciado	
3.2 Ejemplos resueltos (verificados con la fórmula y, en el primero, con expansión del árbol)	
4. Mergesort	
4.1 Pseudocódigo	
4.2 Recurrencia y por qué es $O(n \log n)$	
4.3 Demostración de correctitud	
4.4 Por qué es $O(n \log n)$ — verificación con código	
5. Conteo de inversiones (Práctico, semana 7, Ejercicio 2 — variante “inversión significativa”)	
6. Karatsuba (multiplicación rápida de enteros) y Closest Pair of Points	
6.1 Karatsuba	
6.2 Closest Pair of Points	
7. Trampas típicas de examen	

Resumen ultra-rápido (2 minutos)

.....

Divide y Vencerás

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 5, PROGRAMACIÓN 3 (FING UDELAR)

FUENTE: KLEINBERG & TARDOS, *ALGORITHM DESIGN*, CAP. 5 (+
REFERENCIAS A 13.5) — PRÁCTICOS SEMANA 7 Y 8

0. Por qué existe este tema

Casi todos los problemas que ya viste (grafos, algoritmos ávidos) se resuelven “de una pasada”: recorrés la entrada una vez, tomás decisiones locales, listo. Divide y Vencerás (D&V) ataca un tipo de problema distinto: uno donde **partir la entrada en pedazos, resolver cada pedazo por separado (recursivamente) y después combinar las soluciones parciales** termina siendo más barato que atacar el problema completo de una.

La pregunta que importa en el examen no es “¿qué algoritmo de D&V existe para este problema?” — eso ya te lo van a dar o lo vas a tener que inventar con una pista — sino:

1. **¿Cuál es la recurrencia de tiempo** que describe el algoritmo, y por qué es esa y no otra (¿en cuántos subproblemas se divide? ¿de qué tamaño queda cada uno? ¿cuánto cuesta combinar?).
2. **¿Cómo se resuelve esa recurrencia** para obtener una cota asintótica cerrada — a mano, sin calculadora de recurrencias.
3. **¿Cómo se demuestra la corrección** del algoritmo — casi siempre por inducción fuerte sobre el tamaño de la entrada, con una hipótesis inductiva que dice “el algoritmo funciona bien en instancias estrictamente más chicas”.

Este módulo (cap. 5 del libro + prácticos semana 7-8) importa especialmente porque introduce el patrón de razonamiento —plantear $T(n)$, resolverla, probar corrección por inducción fuerte— que después reaparece, con variantes, en Programación Dinámica (Módulo 6) y en los algoritmos aleatorizados (13.5).

Idea de fondo que se repite en TODO el módulo: en Programación 3 (a diferencia de, por ejemplo, NP-completitud), la fuerza bruta casi siempre **ya es polinomial**. D&V no convierte algo exponencial en polinomial — convierte un polinomio de grado alto (típicamente $O(n^2)$) en uno de grado más bajo (típicamente $O(n \log n)$). Tenerlo claro evita el error de creer que “usar D&V” es sinónimo de “hacer algo tratable”: acá ya era tratable, solo que lento.

1. Esquema general: dividir, resolver, combinar

Un algoritmo de Divide y Vencerás sigue siempre esta estructura de tres pasos:

1. **Dividir (Divide):** partir la instancia de tamaño n en q subinstancias (usualmente $q=2$, pero no siempre), típicamente de tamaño n/b cada una.
2. **Vencer/resolver (Conquer):** resolver cada subinstancia recursivamente (llamando al mismo algoritmo sobre una entrada más chica). El caso base ocurre cuando la subinstancia ya es de tamaño constante (o suficientemente chica para resolverla directamente).
3. **Combinar (Combine):** a partir de las q soluciones parciales, construir la solución de la instancia original.

¿Cuándo conviene D&V sobre un algoritmo directo? Cuando dividir y combinar es *barato* (típicamente lineal, $O(n)$) y, aun así, la recursión termina ahorrando trabajo global respecto de atacar todo de una — porque evita recalcular sobre el conjunto completo en cada “pasada”. El ejemplo canónico: contar inversiones por fuerza bruta es $O(n^2)$ (mirar cada par), pero divide-y-vencerás lo baja a $O(n \log n)$ reutilizando el orden ya calculado en las mitades (Sección 5).

Regla práctica para **reconocer** cuándo un problema es candidato a D&V en un examen: si el enunciado dice “dé un algoritmo con complejidad $O(n \log n)$ ” (o cualquier cota “extraña”, tipo $O(n^{1.59})$ o $O(\log n)$ consultas) para un problema donde la fuerza bruta obvia es $O(n^2)$ o peor, es casi seguro D&V. Si la cota pedida es $O(n)$ plano y el problema tiene estructura de subproblemas independientes con combinación barata, también puede ser D&V con $q=1$ (ver Sección 3).

2. Cómo plantear la recurrencia de tiempo $T(n)$

Dado el pseudocódigo (o el diseño) de un algoritmo de D&V, la recurrencia se arma respondiendo tres preguntas:

- **¿En cuántos subproblemas se divide la entrada?** — llámalo q (o a , según la notación).
- **¿De qué tamaño queda cada subproblema?** — típicamente n/b , con b constante ($b=2$ es lo más común: mitades iguales).
- **¿Cuánto cuesta el trabajo fuera de las llamadas recursivas** (dividir la entrada + combinar los resultados)? — llámalo $f(n)$ (en el caso más común, $f(n)=O(n)$, lineal).

Con esto, la recurrencia queda:

$$T(n) \leq q \cdot T(n/b) + f(n), \text{ con un caso base } T(O(1)) \leq c.$$

Ejemplo 2.1 — Mergesort. Se divide la lista en 2 mitades ($q=2$), cada mitad tiene tamaño $n/2$ ($b=2$), y mezclar (merge) las dos mitades ya ordenadas cuesta $O(n)$. Recurrencia: $T(n) \leq 2T(n/2) + O(n)$.

Ejemplo 2.2 — Karatsuba (multiplicación de enteros, Sección 6). Se parte cada número de n bits en una mitad alta y una mitad baja ($n/2$ bits cada una), y el truco algebraico logra resolver el producto con solo **3** multiplicaciones recursivas de números de $n/2$ bits (en vez de las 4 “obvias”), más $O(n)$ de sumas/restas para combinar. Recurrencia: $T(n) \leq 3T(n/2) + O(n)$.

Error frecuente de tolerancia cero — plantear mal la recurrencia: confundir *cantidad* de subproblemas (q) con el *tamaño* de cada uno (n/b). Si un algoritmo divide la entrada en 4 partes de tamaño $n/2$ (como la versión “ingenua” de multiplicar enteros partiendo en 2×2 y haciendo las 4 multiplicaciones $x_1y_1, x_1y_0, x_0y_1, x_0y_0$), la recurrencia es $T(n)=4T(n/2)+O(n)$ — **no** $T(n)=2T(n/4)+O(n)$. Son recurrencias con soluciones completamente distintas (la primera da $\Theta(n^2)$, como se verá en la Sección 3; confundirla con la segunda da una cota totalmente errónea).

Error frecuente de tolerancia cero — olvidar el costo de “combinar”: es tentador escribir $T(n) = qT(n/b)$ sin el término $f(n)$, como si dividir y combinar fueran gratis. En Mergesort, por ejemplo, el paso de *merge* (fusionar dos listas ordenadas en una) es exactamente lo que hace que el algoritmo funcione — omitirlo de la recurrencia no solo está mal analíticamente, sino que esconde la parte del algoritmo que realmente hace el trabajo “no trivial”.

3. El Teorema Maestro

3.1 ENUNCIADO

Sea una recurrencia de la forma

$$T(n) = a \cdot T(n/b) + f(n), \text{ con } a \geq 1, b > 1 \text{ constantes, y } f(n) \text{ asintóticamente positiva.}$$

(a = cantidad de subproblemas; n/b = tamaño de cada subproblema; $f(n)$ = costo de dividir + combinar. En la notación del libro de Kleinberg-Tardos esto es exactamente su clase de recurrencias $T(n) \leq qT(n/2) + O(n)$, es decir, el caso particular $b=2$, $f(n)=O(n)$ — el libro no plantea el Teorema Maestro en su forma general con b arbitrario y $f(n)$ arbitraria, pero **todo lo que hace en la**

Sección 5.2 (resolver por “desenrollado” $q=1$, $q=2$, $q>2$) es exactamente una demostración manual, caso por caso, del Teorema Maestro para $b=2$. Acá se da la versión general porque es la forma estándar en que aparece en otros cursos y en exámenes de este mismo curso.)

Comparamos $f(n)$ contra $n^{\log_b(a)}$ (la cota que tendría el trabajo hecho **solo en las hojas** del árbol de recursión, si no hubiera nada de trabajo adicional en los niveles intermedios):

Caso	Condición	Conclusión
1	$f(n) = O(n^{\log_b(a) - \epsilon})$ para algún $\epsilon > 0$ ($f(n)$ crece estrictamente más lento que $n^{\log_b a}$, polinomialmente)	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a})$ ($f(n)$ crece igual que $n^{\log_b a}$, a lo sumo con factores log adicionales — la forma más simple del caso 2 es $f(n) = \Theta(n^{\log_b a})$ exacto)	$T(n) = \Theta(n^{\log_b a} \cdot \log n)$
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ para algún $\epsilon > 0$ ($f(n)$ crece estrictamente más rápido), y además se cumple la condición de regularidad: $a \cdot f(n/b) \leq c \cdot f(n)$ para alguna constante $c < 1$ y n suficientemente grande	$T(n) = \Theta(f(n))$

Intuición de por qué son 3 casos: en el árbol de recursión, el trabajo total se reparte entre el nivel 0 (raíz, $f(n)$) y las hojas (nivel $\log_b n$, con $a^{\log_b n} = n^{\log_b a}$ hojas de trabajo constante cada una). El Teorema Maestro pregunta: **¿quién domina, la raíz o las hojas?** Caso 1: dominan las hojas (el trabajo por nivel *crece* a medida que se baja en el árbol — esto es exactamente lo que KT llama el caso $q>2$, con $\log_2 q > 1$). Caso 3: domina la raíz (el trabajo por nivel *decrece* — el caso $q=1$ de KT, o el caso $q=2$ con $f(n) = O(n^2)$ de la ecuación (5.6) del libro). Caso 2: **empate** en cada nivel (el “filo de la navaja” que menciona KT explícitamente para $q=2$ con $f(n) = O(n)$ — cada uno de los $\log n$ niveles aporta lo mismo, de ahí el factor $\log n$ extra).

Error frecuente de tolerancia cero — aplicar el Teorema Maestro fuera de sus condiciones: el teorema **no cubre todos los casos**. En particular falla (no aplica ninguno de los 3 casos, y hay que resolver “a mano”) cuando:

- $f(n)$ no es polinomialmente distinta de $n^{\log_b a}$ en ninguna dirección — por ejemplo si hay un “hueco” logarítmico que no es ni Θ exacto ni una diferencia polinomial genuina.

Ejemplo clásico: $T(n) = 2T(n/2) + n \log n$. Acá $f(n) = n \log n$ y $n^{\log_b a} = n^{\log_2 2} = n$. $f(n)$ es *más grande* que n , pero **no** por un factor polinomial n^ϵ — es solo un factor $\log n$. No cae en el caso 3 (no hay $\epsilon > 0$ tal que $n \log n = \Omega(n^{1+\epsilon})$) ni en el caso 2 ($n \log n \neq \Theta(n)$). Hay que resolverlo por sustitución/desenrollado directo (da $T(n) = \Theta(n \log^2 n)$).

- En el caso 3, si **no se verifica la condición de regularidad** — es una condición técnica que casi siempre se cumple cuando $f(n)$ es un polinomio, pero **no** es automática y debe mencionarse explícitamente en una demostración rigurosa.
- Cuando a , b , o los tamaños de subproblema **no son constantes** (por ejemplo, algoritmos donde el tamaño del subproblema depende del valor de la entrada, no solo de n) — el Teorema Maestro asume una estructura de recursión “regular”, uniforme en todos los niveles.

3.2 EJEMPLOS RESUELTOS (VERIFICADOS CON LA FÓRMULA Y, EN EL PRIMERO, CON EXPANSIÓN DEL ÁRBOL)

Ejemplo 3.1 — $T(n) = 2T(n/2) + n$ (recurrencia de Mergesort).

$$a=2, b=2 \rightarrow n^{\log_b a} = n^{\log_2 2} = n^1 = n.$$

$$f(n) = n = \Theta(n^1) = \Theta(n^{\log_b a}) \rightarrow \text{caso 2.}$$

$$\text{Conclusión: } T(n) = \Theta(n \log n).$$

Verificación por expansión del árbol de recursión (a mano):

Nivel 0: 1 problema de tamaño n , trabajo cn .

Nivel 1: 2 problemas de tamaño $n/2$, trabajo $2 \cdot c(n/2) = cn$.

Nivel 2: 4 problemas de tamaño $n/4$, trabajo $4 \cdot c(n/4) = cn$.

Nivel j : 2^j problemas de tamaño $n/2^j$, trabajo $2^j \cdot c(n/2^j) = cn$ (constante en *todos* los niveles — el “filo de la navaja”).

Hay $\log_2 n$ niveles (hasta que el tamaño llega a 1). Total: $cn \cdot \log_2 n = \Theta(n \log n)$. ✓ Coincide exactamente con la fórmula del caso 2.

Ejemplo 3.2 — $T(n) = 4T(n/2) + n$ (versión “ingenua” de multiplicar enteros, 4 llamadas en vez de 3).

$$a=4, b=2 \rightarrow n^{\log_b a} = n^{\log_2 4} = n^2.$$

$$f(n) = n = O(n^{2-\varepsilon}) \text{ tomando } \varepsilon=1 \text{ (} n \text{ crece estrictamente más lento que } n^2 \text{)} \rightarrow \text{caso 1.}$$

$$\text{Conclusión: } T(n) = \Theta(n^2).$$

Verificación numérica rápida: con $q=4$ la fórmula de KT (5.4) da exactamente

$O(n^{\log_2 q}) = O(n^{\log_2 4}) = O(n^2)$ — coincide con el caso 1 del Teorema Maestro (KT deriva este resultado “a mano” con suma geométrica creciente, sin nombrarlo Teorema Maestro, pero es el mismo resultado).

Ejemplo 3.3 — $T(n) = 3T(n/2) + n$ (Karatsuba, multiplicación de enteros).

$$a=3, b=2 \rightarrow n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}.$$

$f(n) = n = O(n^{1.585-\varepsilon})$ para, por ejemplo, $\varepsilon=0.5 \rightarrow$ **caso 1**.

Conclusión: $T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585}) \approx O(n^{1.59})$.

Este es exactamente el resultado (5.13) del libro para `Recursive-Multiply` (Sección 6 de esta guía).

Ejemplo 3.4 — $T(n) = 2T(n/2) + n^2$ (recurrencia (5.6) del libro, ecuación de recombinación cuadrática — aparece en alineamiento de secuencias con espacio lineal, Capítulo 6).

$$a=2, b=2 \rightarrow n^{\log_b a} = n.$$

$f(n) = n^2 = \Omega(n^{1+1})$, tomando $\varepsilon=1 \rightarrow$ posible **caso 3**. Falta verificar regularidad: $a \cdot f(n/b) = 2 \cdot (n/2)^2 = n^2/2 \leq c \cdot n^2$ con $c=1/2 < 1 \checkmark$ (se cumple para todo n).

Conclusión: $T(n) = \Theta(n^2)$.

Contraste con la intuición ingenua: el libro señala explícitamente que uno esperaría $T(n)=O(n^2 \log n)$ “por analogía” con Mergesort — pero como el trabajo por nivel *decrece* geométricamente ($cn^2, cn^2/2, cn^2/4, \dots$) en vez de mantenerse constante, la suma converge a $O(n^2)$ sin el factor $\log n$ extra. Esto es precisamente el caso 3 del Teorema Maestro: domina la raíz.

Truco de verificación rápida: antes de aplicar cualquiera de los 3 casos, calculá primero $\log_b a$ (con calculadora o de memoria: $\log_2 2=1, \log_2 3 \approx 1.585, \log_2 4=2$) y compará el *grado polinomial* de $f(n)$ contra ese número. Si $f(n)$ es un polinomio n^k (con o sin logs), la comparación es directa: $k < \log_b a \rightarrow$ caso 1; $k = \log_b a$ (sin logs extra) $\rightarrow$ caso 2; $k > \log_b a \rightarrow$ caso 3 (y ahí sí, chequeá regularidad).

4. Mergesort

4.1 PSEUDOCÓDIGO

```

Sort-and-Count(L):
    si |L| ≤ 1: devolver (L, 0)           // 0 inversiones, ya ordenada
    m = ⌊n/2⌋
    A = primeros m elementos de L
    B = elementos restantes de L
    (A_ord, rA) = Sort-and-Count(A)
    (B_ord, rB) = Sort-and-Count(B)
    (L_ord, r) = Merge-and-Count(A_ord, B_ord)
    devolver (L_ord, rA + rB + r)

Merge-and-Count(A, B):    // A, B ya ordenadas ascendentemente
    C = []; i = j = 0; count = 0
    mientras A y B no vacías:
        si A[i] ≤ B[j]: agregar A[i] a C; i++
        si no: agregar B[j] a C; j++; count += (elementos restantes en A)
    agregar el resto de A o B (la que no se vació) a C
    devolver (C, count)

```

(Acá se muestra la versión “Sort-and-Count” que además cuenta inversiones, porque es la que reutilizamos en la Sección 5; el Mergesort “puro” es exactamente igual, sin la variable `count`.)

4.2 RECURRENCIA Y POR QUÉ ES $O(N \log N)$

Dividir en dos mitades es $O(1)$ (o $O(n)$ si se copian los datos), combinar con `Merge` es $O(n)$ (recorre A y B una sola vez, cada elemento se agrega a C exactamente una vez). Por lo tanto:

$$T(n) \leq 2T(n/2) + O(n), \quad T(2) \leq c.$$

Por el Teorema Maestro (caso 2, Ejemplo 3.1) o por la expansión directa del árbol (verificada arriba), $T(n) = \Theta(n \log n)$.

4.3 DEMOSTRACIÓN DE CORRECTITUD

Afirmación: `Sort-and-Count(L)` devuelve L ordenada ascendentemente, para toda lista L.

Demostración (inducción fuerte en $n = |L|$):

Caso base ($n \leq 1$): una lista de 0 o 1 elementos está trivialmente ordenada. El algoritmo la devuelve sin modificar. ✓

Paso inductivo: sea $n > 1$, y supongamos (hipótesis inductiva) que el algoritmo ordena correctamente toda lista de tamaño estrictamente menor que n . `Sort-and-Count(L)` divide L en A (tamaño $\lfloor n/2 \rfloor < n$ cuando $n > 1$) y B (tamaño $\lfloor n/2 \rfloor < n$). Por hipótesis inductiva, `A_ord` y `B_ord` son A y B correctamente ordenadas.

Queda por verificar que `Merge-and-Count(A_ord, B_ord)` produce la lista ordenada que resulta de fusionar `A_ord` y `B_ord` — este es el **invariante de “mitades ordenadas + merge correcto”** que menciona el enunciado: en cada iteración del `mientras`, el algoritmo compara los elementos actuales de A y B (los más chicos de cada lista aún no colocados) y mueve a C el menor de los dos. Como A y B están ordenadas, el elemento movido a C es, en cada paso, el menor elemento entre **todos** los que quedan por colocar (en A o en B) — porque cualquier elemento de A no considerado todavía es $\geq A[i]$ (A está ordenada) y cualquiera de B es $\geq B[j]$. Por lo tanto C se construye en orden estrictamente no decreciente, y al terminar (cuando una de las dos listas se vació) el resto de la otra lista, ya ordenada, se agrega al final sin violar el orden (porque todo elemento restante en la lista no vaciada es $\geq$ el último elemento ya puesto en C — es exactamente el mismo argumento). Así, C es L ordenada. ■

4.4 POR QUÉ ES $O(N \log N)$ — VERIFICACIÓN CON CÓDIGO

Verificación con Python (Mergesort + conteo de inversiones, contrastado contra fuerza bruta $O(n^2)$):

```

def merge_and_count(A, B):
    C, i, j, count = [], 0, 0, 0
    while i < len(A) and j < len(B):
        if A[i] <= B[j]:
            C.append(A[i]); i += 1
        else:
            C.append(B[j]); j += 1
            count += len(A) - i          # todos los que quedan en A forman inversion
    con B[j]
    C.extend(A[i:]); C.extend(B[j:])
    return C, count

def sort_and_count(L):
    if len(L) <= 1:
        return L, 0
    m = len(L)//2
    A, rA = sort_and_count(L[:m])
    B, rB = sort_and_count(L[m:])
    C, r = merge_and_count(A, B)
    return C, rA + rB + r

def brute_inversions(L):
    n = len(L)
    return sum(1 for i in range(n) for j in range(i+1, n) if L[i] > L[j])

# 200 pruebas aleatorias, tamaños 1..40 -> todas coinciden con fuerza bruta O(n^2)

```

Resultado real de la corrida (200 casos aleatorios, n entre 1 y 40): ☐ OK en todos — el orden de salida coincide con `sorted(L)` y el conteo de inversiones coincide exactamente con la fuerza bruta $O(n^2)$ en cada uno de los 200 casos.

Ejemplo puntual verificado: $L=[2,4,1,3,5] \rightarrow$ ordenada $[1,2,3,4,5]$, 3 inversiones — coincide con el Ejemplo 5.4 del libro (pares $(2,1),(4,1),(4,3)$).

5. Conteo de inversiones (Práctico, semana 7, Ejercicio 2 — variante “inversión significativa”)

Este es el ejercicio 2 de la semana 7 (KT Ex. 5.2): en vez de contar pares $i < j$ con $L_i > L_j$ (inversión “normal”), se cuenta pares con $L_i > 2 \cdot L_j$ (“inversión significativa”). La resolución completa está en el archivo `Practico5_Resolucion.md`; acá se deja solo el punto clave de D&V y la verificación numérica, porque es la aplicación más directa de todo lo visto en la Sección 4.

Idea: el mismo esquema Sort-and-Count / Merge-and-Count funciona, cambiando la condición de comparación en el conteo (no en el merge, que sigue comparando $A[i] \leq B[j]$ normalmente para producir la salida ordenada). Se usa un puntero adicional que avanza monótonamente (nunca retrocede) para contar, en $O(n)$ por nivel, cuántos pares $(A[i], B[j])$ cumplen $A[i] > 2 \cdot B[j]$.

Verificación numérica (200 pruebas aleatorias, comparado contra fuerza bruta $O(n^2)$):

el algoritmo adaptado (puntero j que solo avanza) coincidió exactamente con

`brute_sig(L) = sum(1 for i 2*L[j])` en los 200 casos.

Ejemplo puntual: $L = [9, 1, 3, 8, 2] \rightarrow 4$ inversiones significativas, verificado tanto por el algoritmo D&V como por fuerza bruta: los pares son $(9,1)[9>2]$, $(9,3)[9>6]$, $(9,2)[9>4]$, $(8,2)[8>4]$.

Por qué el puntero puede “solo avanzar” (y por eso el conteo sigue siendo $O(n)$ por nivel, preservando la recurrencia $T(n)=2T(n/2)+O(n)$): a medida que i avanza en A ($A[i]$ creciente, porque A está ordenada), el umbral $2 \cdot B[j]$ necesario para que deje de cumplirse $A[i]>2B[j]$ solo puede requerir que j avance también (nunca retroceder), porque B también está ordenada. Este es exactamente el argumento que generaliza Merge-and-Count del libro (que cuenta $A[i]>B[j]$ con la misma técnica de puntero monótono) al caso $A[i]>2B[j]$.

6. Karatsuba (multiplicación rápida de enteros) y Closest Pair of Points

El material del curso (capítulo 5 del libro) cubre ambas aplicaciones clásicas de D&V más allá de ordenar/contar inversiones. Se presentan ambas brevemente porque las dos aparecen en la Sección 5.5 y 5.4 del libro respectivamente, con recurrencias que ilustran los tres casos del Teorema Maestro.

6.1 KARATSUBA

Problema: multiplicar dos enteros de n bits. El algoritmo “de la escuela” (sumar productos parciales) es $O(n^2)$.

Idea D&V: escribir $x = x_1 \cdot 2^{n/2} + x_0$ (mitad alta, mitad baja), $y = y_1 \cdot 2^{n/2} + y_0$. Entonces $xy = x_1 y_1 \cdot 2^n + (x_1 y_0 + x_0 y_1) \cdot 2^{n/2} + x_0 y_0$. La versión ingenua necesita **4** multiplicaciones recursivas ($x_1 y_1$, $x_1 y_0$, $x_0 y_1$, $x_0 y_0$) $\rightarrow T(n)=4T(n/2)+O(n) \rightarrow \Theta(n^2)$ (Ejemplo 3.2, no mejora nada).

El truco de Karatsuba: calcular $p = (x_1 + x_0)(y_1 + y_0)$ (una sola multiplicación), más $x_1 y_1$ y $x_0 y_0$ (dos multiplicaciones más). El término cruzado se obtiene como $p - x_1 y_1 - x_0 y_0$ (sin una cuarta multiplicación, solo restas $O(n)$). Total: **3** multiplicaciones recursivas.

```

Recursive-Multiply(x, y):
    si x, y son de 1 bit: devolver x·y
    escribir  $x = x_1 \cdot 2^{(n/2)} + x_0$ ,  $y = y_1 \cdot 2^{(n/2)} + y_0$ 
    p      = Recursive-Multiply( $x_1 + x_0$ ,  $y_1 + y_0$ )
     $x_1 y_1$  = Recursive-Multiply( $x_1$ ,  $y_1$ )
     $x_0 y_0$  = Recursive-Multiply( $x_0$ ,  $y_0$ )
    devolver  $x_1 y_1 \cdot 2^n + (p - x_1 y_1 - x_0 y_0) \cdot 2^{(n/2)} + x_0 y_0$ 

```

Recurrencia: $T(n) \leq 3T(n/2) + O(n) \rightarrow$ por el Teorema Maestro (Ejemplo 3.3, caso 1) $\rightarrow T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585})$, resultado (5.13) del libro.

Verificación con Python (500 pruebas, enteros de hasta 12 dígitos):

```

def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y
    n = max(len(str(x)), len(str(y)))
    m = n // 2
    hx, lx = divmod(x, 10**m)
    hy, ly = divmod(y, 10**m)
    z0 = karatsuba(lx, ly)
    z2 = karatsuba(hx, hy)
    z1 = karatsuba(lx + hx, ly + hy) - z0 - z2
    return z2 * 10**(2*m) + z1 * 10**m + z0

```

Resultado: OK en las 500 pruebas contra la multiplicación nativa de Python. Ejemplo puntual: $1234 \times 5678 = 7006652$, coincide con `karatsuba(1234, 5678)`.

Error frecuente: pensar que como Karatsuba usa 3 llamadas recursivas de tamaño $n/2$ (en vez de las 2 de Mergesort), el algoritmo debería ser “un tercio más lento” o algo lineal en la diferencia. En realidad el exponente cambia de forma no lineal: $\log_2 3 \approx 1.585$ no es “2 – algo chico”, es una mejora real y sustancial sobre el n^2 ingenuo, aunque sigue siendo peor que $O(n \log n)$ — no hay que confundir “mejoró” con “es igual de bueno que Mergesort”.

6.2 CLOSEST PAIR OF POINTS

Problema: dados n puntos en el plano, hallar el par más cercano (distancia euclídea mínima). Fuerza bruta: $O(n^2)$ (todas las $C(n,2)$ distancias).

Idea D&V: ordenar los puntos por x e y una sola vez ($O(n \log n)$), dividir por una recta vertical en mitad izquierda Q y mitad derecha R ($n/2$ puntos cada una), resolver recursivamente en cada mitad obteniendo δ = mínima distancia encontrada en Q o en R . El paso difícil es la **combinación**: puede existir un par $(q \in Q, r \in R)$ más cercano que δ , y hay que encontrarlo en $O(n)$, no en $O(n^2)$.

Hecho clave (demostrado en el libro, (5.10)): solo hace falta mirar los puntos dentro de una franja de ancho 2δ alrededor de la recta divisoria, ordenados por y — y , dentro de esa franja, **cada punto solo puede tener un par más cercano que δ entre los siguientes 15 puntos en el orden de y** (se prueba por un argumento de “cajas” de lado $\delta/2$: cada caja contiene a lo sumo 1 punto de la franja, porque si tuviera 2 estarían a distancia $< \delta$ y contradirían la definición de δ ; y cualquier par a más de 15 posiciones de distancia en la lista ordenada por y está separado por al menos 3 filas de cajas, o sea a distancia $\geq 3\delta/2 > \delta$).

Esto acota el trabajo de combinación en $O(n)$ (constante por punto), dando la recurrencia $T(n) \leq 2T(n/2) + O(n) \rightarrow \Theta(n \log n)$ (más el $O(n \log n)$ del ordenamiento inicial, que no domina).

Verificación con Python (100 pruebas, hasta 60 puntos, comparado contra fuerza bruta $O(n^2)$): implementación de `closest_pair_rec` siguiendo exactamente el algoritmo del libro (incluida la revisión de “hasta 15 vecinos siguientes en la franja”). Resultado: `OK` en las 100 pruebas — la distancia mínima al cuadrado hallada por D&V coincide con la de fuerza bruta en todos los casos.

7. Trampas típicas de examen

1. Confundir cantidad de subproblemas con tamaño de cada uno. Ya visto en la Sección 2 — releer el error frecuente ahí. Antes de escribir $T(n)=aT(n/b)+f(n)$, señalará explícitamente en tu borrador “ $a =$ subproblemas, cada uno de tamaño $n/$ ”.

2. Aplicar el Teorema Maestro cuando $f(n)$ no es “polinomialmente comparable” a $n^{\log_b a}$. El ejemplo típico de examen es $T(n)=2T(n/2)+n \log n$ (que aparece, por ejemplo, en la complejidad de construir el árbol de FFT con ciertas implementaciones, o en variantes de Mergesort con comparaciones más caras). Ningún caso del Teorema Maestro aplica directamente — hay que resolver por sustitución (adivinar $T(n)=\Theta(n \log^2 n)$ y verificar por inducción) o por expansión del árbol.

3. Olvidar demostrar — o mencionar — la condición de regularidad en el caso 3. Si en un examen aplicás el caso 3 sin decir una palabra sobre por qué $af(n/b) \leq cf(n)$, un corrector

estricto puede (con razón) no dar el ejercicio por completo, porque el caso 3 es el único de los tres que tiene una hipótesis extra explícita.

4. Ignorar los pisos y techos ($\lfloor n/2 \rfloor$ vs $\lceil n/2 \rceil$) cuando n no es potencia de 2, y no justificar por qué no importa. El propio libro señala esto explícitamente en la Sección 5.1: se puede ignorar en el análisis asintótico (no cambia el orden de crecimiento), pero un enunciado riguroso lo menciona al menos una vez y explica por qué se puede simplificar sin pérdida de generalidad.

5. Plantear la demostración de corrección “mirando la recursión desde afuera” en vez de por inducción fuerte real. La corrección de un algoritmo D&V casi siempre se prueba con: caso base explícito + hipótesis inductiva (“el algoritmo funciona correctamente en toda instancia de tamaño estrictamente menor a n ”) + paso inductivo que usa esa hipótesis sobre las subinstancias generadas, y luego demuestra que el paso de *combinar* preserva la propiedad deseada. Saltarse la hipótesis inductiva explícita (como en el Ejercicio 2(b) del práctico de semana 7, que la pide con la fórmula $R(L)=R(A)+r+R(B)$ explícita) es el error de forma más común y más penalizado.

6. Confundir “ $T(n) = O(f(n))$ ” con “ $T(n) = \Theta(f(n))$ ” al enunciar el resultado del Teorema Maestro. Los 3 casos del teorema dan cotas **ajustadas (Θ)**, no solo cotas superiores. Si el enunciado del examen pide “cota superior” alcanza con $O(\cdot)$, pero si pide “orden de crecimiento” o “tiempo de ejecución” sin calificar, se espera Θ .

Resumen ultra-rápido (2 minutos)

- **D&V = dividir + resolver recursivamente + combinar.** Conviene cuando dividir/combinar es barato (usualmente $O(n)$) y reduce el grado polinomial de la solución.
- **Plantear $T(n)$:** identificar **a** (cantidad de subproblemas) y **n/b** (tamaño de cada uno) y **$f(n)$** (costo de dividir+combinar). $T(n) = aT(n/b) + f(n)$.
- **Teorema Maestro** (comparar $f(n)$ contra $n^{\log_b a}$): $f(n)$ más chico polinomialmente $\rightarrow \Theta(n^{\log_b a})$ (caso 1, “dominan las hojas”); igual (Θ exacto) $\rightarrow \Theta(n^{\log_b a} \log n)$ (caso 2, “empate en cada nivel”); $f(n)$ más grande polinomialmente + regularidad $\rightarrow \Theta(f(n))$ (caso 3, “domina la raíz”). Si no hay comparación polinomial clara (ej. factor log de diferencia), no aplica — resolver por sustitución/árbol a mano.
- **Mergesort:** $T(n)=2T(n/2)+O(n) \rightarrow \Theta(n \log n)$. Corrección por inducción fuerte: hipótesis “ordena correctamente instancias $<n$ ”, combinar con merge preserva el orden porque en cada paso se mueve el mínimo global de lo que resta.

- **Conteo de inversiones:** mismo esquema que Mergesort, contando en el merge cuántos elementos quedan en A cuando se extrae de B ($O(1)$ amortizado por elemento). Generaliza directo a variantes como “inversión significativa” ($A[i] > 2B[j]$) con un segundo puntero monótono.
- **Karatsuba:** $T(n) = 3T(n/2) + O(n) \rightarrow \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$, usando el truco de $(x_1 + x_0)(y_1 + y_0)$ para evitar la 4ª multiplicación.
- **Closest pair:** $T(n) = 2T(n/2) + O(n) \rightarrow \Theta(n \log n)$, con el paso clave de “franja de ancho 2δ + a lo sumo 15 vecinos relevantes por punto” para combinar en $O(n)$.
- **Trampas de examen:** confundir a con n/b ; aplicar Master fuera de sus condiciones (sobre todo con $f(n)$ con factores log “suelos”); olvidar la regularidad en el caso 3; olvidar la hipótesis inductiva explícita al probar corrección; confundir O con Θ en la conclusión.